



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERIA
Escuela Profesional de Ingeniería de Sistemas

INFORME DE LABORATORIO
"LINQ ARREGLO"

Curso: Sistemas de Información

Docente: Ing. Breno Luque Zuñiga

Peralta Sebán, Daniel Sebastián (2024016803)

Tacna – Perú
2026



INDICE

I. INFORMACIÓN GENERAL	3
- Objetivos	3
- Equipos, materiales, programas y recursos utilizados	3
II. MARCO TEORICO	3
III. PROCEDIMIENTO	4
Programa: Catálogo de Productos y Buscador:.....	4
1. Diseño del Programa.....	4
2. Tabla de Controles.....	4
3. Código del Programa.....	5
IV. ANALISIS E INTERPRETACION DE RESULTADOS	9
CONCLUSIONES	10
RECOMENDACIONES	10
WEBGRAFIA	10



INFORME DE LABORATORIO

TEMA: LINQ ARREGLO

I. INFORMACIÓN GENERAL

- **Objetivos:**
 - Crear una aplicación para gestionar un catálogo de productos.
 - Implementar un buscador que combine texto e incluso categorías.
 - Entender el uso de listas como formato de almacenamiento de datos.
 - Aplicar LINQ para filtrar datos.
 - Tratar imágenes dentro de aplicaciones Windows Forms.
 - Permitir la interacción del usuario con los datos utilizando selecciones y eventos.
- **Equipos, materiales, programas y recursos utilizados:**
 - Computadora con sistema operativo Windows 8
 - Visual Studio
 - Lenguaje de programación C#.
 - .NET Framework

II. MARCO TEORICO

List: Es una colección dinámica que permite almacenar elementos de un mismo tipo, con capacidad de crecer según sea necesario.

LINQ (Language Integrated Query): LINQ permite realizar consultas sobre colecciones de datos de forma sencilla y eficiente utilizando expresiones lambda.

DataGridView: Permite visualizar datos en forma tabular, facilitando la interacción con el usuario.

Manejo de imágenes: Las imágenes pueden cargarse desde archivos y asignarse a controles como PictureBox o columnas de tipo imagen en DataGridView.

Eventos en Windows Forms: Permiten ejecutar acciones cuando el usuario interactúa con la interfaz (clics, selección, etc.).

III. PROCEDIMIENTO

Programa: Catálogo de Productos y Buscador:

1. Diseño del Programa

Catálogo de Productos y Buscador

TIPO DE PRODUCTO

Panel de Búsqueda y Filtros

Buscar Producto:

	Código	Imagen	Nombre del Producto	Categoría	Marca	Precio	Stc
▶	0000001		Smart TV LG 50"	Electrónica	LG	1800	5
	0000002		Laptop HP Pavilion	Computadoras	HP	3500	8
	0000003		Mouse Logitech	Periféricos	Logitech	80	20
	0000004		SSD 1TB Kingston	Almacenamiento	Kingston	300	15
	0000005		Audifonos Sony	Audio y Video	Sony	200	10
	0000006		Microondas Samsung	Hogar	Samsung	500	6
	0000007		Escritorio Oficina	Oficina	Halion	700	4

Detalles del Producto Seleccionado

 Nombre: Marca:

2. Tabla de Controles

Control	Propiedad	Valor
Label1	Text	Catálogo de Productos y Buscador
	Font	bold type
Label2	Text	TIPO DE PRODUCTO
Label3	Text	Panel de Búsqueda y Filtros
Label4	Text	Buscar Producto:
Label5	Text	Detalles del Producto Seleccionado
Label6	Text	Nombre:
Label7	Text	Marca:
Button1	Name	btnBuscar
	Text	BUSCAR
Button2	Name	btnLimpiar
	Text	Limpiar
Button3	Name	btnFicha
	Text	Ver Ficha
Button4	Name	btnAñadir
	Text	Añadir
Textbox1	Name	txtboxBuscar

Textbox2	Name	txtNombre
Textbox3	ReadOnly	True
	Name	txtMarca
	ReadOnly	True
dataGridView1	Name	dataGridView1
ListBox1	Name	listProductos
pictureBox1	Name	pictureBox1

3. Código del Programa

Funcionalidad 1: Definición de la clase Producto

Se utiliza programación orientada a objetos para representar cada producto como una entidad independiente.

Propiedades:

- Código
- Imagen
- Nombre
- Categoría
- Marca
- Precio
- Stock

```

15 referencias
public class Producto
{
    10 referencias
    public string Codigo { get; set; }
    11 referencias
    public Image Imagen { get; set; }
    13 referencias
    public string Nombre { get; set; }
    11 referencias
    public string Categoria { get; set; }
    12 referencias
    public string Marca { get; set; }
    10 referencias
    public double Precio { get; set; }
    10 referencias
    public int Stock { get; set; }
}

```

Funcionalidad 2: Cargar de imágenes

Se implementa una búsqueda en múltiples rutas para asegurar que las imágenes puedan cargarse correctamente.

Método: LoadImage: Verifica si el nombre del archivo es válido.

```

8 referencias
private Image LoadImage(string fileName)
{
    if (string.IsNullOrEmpty(fileName)) return SystemIcons.Question.ToBitmap();

    var baseDir = AppDomain.CurrentDomain.BaseDirectory;
    var attempts = new[] {
        Path.Combine(baseDir, fileName),
        Path.Combine(baseDir, "..", fileName),
        Path.Combine(baseDir, "..", "..", fileName),
        Path.Combine(baseDir, "..", "..", "..", fileName)
    };
}

```

Comprueba si la imagen existe en distintas rutas y se carga la imagen desde archivo.

```
foreach (var p in attempts)
{
    try
    {
        var full = Path.GetFullPath(p);
        if (File.Exists(full))
        {
            using (var fs = File.OpenRead(full))
            {
                return Image.FromStream(fs);
            }
        }
    }
    catch { }
}

return SystemIcons.Question.ToBitmap();
}
```

Funcionalidad 3: Inicialización del sistema

Paso 1: Configuración del DataGridView

Se agregan columnas manualmente para mostrar datos y permite visualizar imágenes dentro de la tabla.

```
1 referencia
private void Form1_Load_1(object sender, EventArgs e)
{
    dataGridView1.Columns.Add(new DataGridViewTextBoxColumn { HeaderText = "Código", DataPropertyName = "Codigo" });

    DataGridViewImageColumn imgCol = new DataGridViewImageColumn();
    imgCol.HeaderText = "Imagen";
    imgCol.DataPropertyName = "Imagen";
    imgCol.ImageLayout = DataGridViewImageCellLayout.Zoom;
    dataGridView1.Columns.Add(imgCol);
    dataGridView1.SelectionChanged += dataGridView1_SelectionChanged;

    dataGridView1.Columns.Add(new DataGridViewTextBoxColumn { HeaderText = "Nombre del Producto", DataPropertyName = "Nombre", Width = 150 });
    dataGridView1.Columns.Add(new DataGridViewTextBoxColumn { HeaderText = "Categoría", DataPropertyName = "Categoría" });
    dataGridView1.Columns.Add(new DataGridViewTextBoxColumn { HeaderText = "Marca", DataPropertyName = "Marca" });
    dataGridView1.Columns.Add(new DataGridViewTextBoxColumn { HeaderText = "Precio", DataPropertyName = "Precio" });
    dataGridView1.Columns.Add(new DataGridViewTextBoxColumn { HeaderText = "Stock", DataPropertyName = "Stock" });
}
```

Paso 2: Carga de productos

Se agregan productos con datos completos y se inicializa el sistema con datos predefinidos listos para ser consultados.

```
// Datos
listaProductos.Add(new Producto { Codigo="0000001", Nombre="Smart TV LG 50\"", Categoria="Electrónica", Marca="LG", Precio=1800, Stock=5, Imagen=LoadImage("Smart TV LG 50.png") });
listaProductos.Add(new Producto { Codigo="0000002", Nombre="Laptop HP Pavilion", Categoria="Computadoras", Marca="HP", Precio=3500, Stock=8, Imagen=LoadImage("Laptop HP Pavilion.png") });
listaProductos.Add(new Producto { Codigo="0000003", Nombre="Mouse Logitech", Categoria="Periféricos", Marca="Logitech", Precio=80, Stock=20, Imagen=LoadImage("Mouse Logitech.png") });
listaProductos.Add(new Producto { Codigo="0000004", Nombre="SSD 1TB Kingston", Categoria="Almacenamiento", Marca="Kingston", Precio=300, Stock=15, Imagen=LoadImage("SSD 1TB Kingston.png") });
listaProductos.Add(new Producto { Codigo="0000005", Nombre="Audífonos Sony", Categoria="Audio y Video", Marca="Sony", Precio=200, Stock=10, Imagen=LoadImage("Audífonos Sony.png") });
listaProductos.Add(new Producto { Codigo="0000006", Nombre="Microondas Samsung", Categoria="Hogar", Marca="Samsung", Precio=500, Stock=6, Imagen=LoadImage("Microondas Samsung.png") });
listaProductos.Add(new Producto { Codigo="0000007", Nombre="Escritorio Oficina", Categoria="Oficina", Marca="Halion", Precio=700, Stock=4, Imagen=LoadImage("Escritorio Oficina.png") });
listaProductos.Add(new Producto { Codigo="0000008", Nombre="Mochila Laptop", Categoria="Accesorios", Marca="Targus", Precio=120, Stock=12, Imagen=LoadImage("Mochila Laptop.png") });

dataGridView1.DataSource = listaProductos;
```

Funcionalidad 4: Búsqueda de productos

Se combinan dos condiciones de búsqueda, logrando un filtrado más preciso. Se obtiene el texto ingresado.

```
private void btnBuscar_Click(object sender, EventArgs e)
{
    string texto = txtboxBuscar.Text?.ToLower() ?? string.Empty;
    string categoria = listProductos.SelectedItem != null
        ? listProductos.SelectedItem.ToString()
        : "TODOS LOS PRODUCTOS";
}
```

Filtra por nombre.

```
var resultado = listaProductos.Where(p =>
    (p.Nombre != null && p.Nombre.ToLower().Contains(texto)) &&
```

Filtra por categoría.

```
(categoria == "TODOS LOS PRODUCTOS" || p.Categoria == categoria)).ToList();

dataGridView1.DataSource = null;
dataGridView1.DataSource = resultado;
}
```

Catálogo de Productos y Buscador

TIPO DE PRODUCTO

- TODOS LOS PRODUCTOS
- Electrónica
- Computadoras
- Periféricos
- Almacenamiento
- Audio y Video
- Hogar
- Oficina
- Accesorios

Panel de Búsqueda y Filtros

Buscar Producto:

	Código	Imagen	Nombre del Producto	Categoría	Marca	Precio	Sto ^
▶	0000001		Smart TV LG 50"	Electrónica	LG	1800	5
	0000002		Laptop HP Pavilion	Computadoras	HP	3500	8
	0000003		Mouse Logitech	Periféricos	Logitech	80	20
	0000004		SSD 1TB Kingston	Almacenamiento	Kingston	300	15
	0000005		Audífonos Sony	Audio y Video	Sony	200	10
	0000006		Microondas Samsung	Hogar	Samsung	500	6
	0000007		Escritorio Oficina	Oficina	Halion	700	4

Detalles del Producto Seleccionado

 Nombre: Marca:

Panel de Búsqueda y Filtros

Buscar Producto:

	Codigo	Imagen	Nombre	Categoria	Marca	Precio	Stock
▶	0000002		Laptop HP Pavilion	Computadoras	HP	3500	8
	0000008		Mochila Laptop	Accesorios	Targus	120	12

Detalles del Producto Seleccionado

 Nombre: Marca:

TIPO DE PRODUCTO

- TODOS LOS PRODUCTOS
- Electrónica
- Computadoras
- Periféricos**
- Almacenamiento
- Audio y Video
- Hogar
- Oficina
- Accesorios

Panel de Búsqueda y Filtros

Buscar Producto:

	Codigo	Imagen	Nombre	Categoria	Marca	Precio	Stock
▶	0000003		Mouse Logitech	Periféricos	Logitech	80	20

Funcionalidad 5: Limpiar búsqueda

Limpia el campo de búsqueda.

```
1 referencia
private void btnLimpiar_Click(object sender, EventArgs e)
{
    txtboxBuscar.Clear();
    listProductos.SelectedIndex = 0;
}
```

Restaura la lista original.

```
dataGridView1.DataSource = null;
dataGridView1.DataSource = listaProductos;
}
```

Funcionalidad 6: Selección de producto

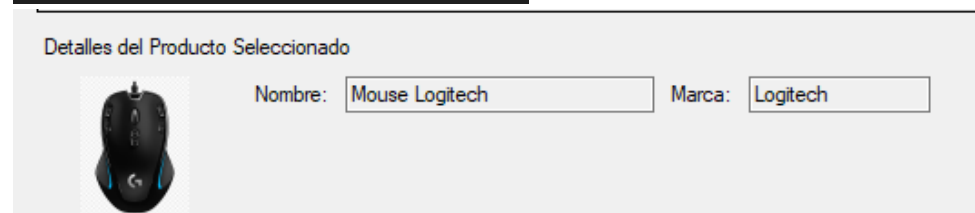
Permite visualizar detalles del producto seleccionado en tiempo real.

Obtiene el producto seleccionado.

```
private void dataGridView1_CellContentClick(object sender, DataGridViewCellEventArgs e)
{
    if (e.RowIndex >= 0)
    {
        var p = (Producto)dataGridView1.Rows[e.RowIndex].DataBoundItem;
    }
}
```

Muestra el nombre, marca y la imagen

```
txtNombre.Text = p.Nombre;
txtMarca.Text = p.Marca;
pictureBox1.Image = p.Imagen;
}
```

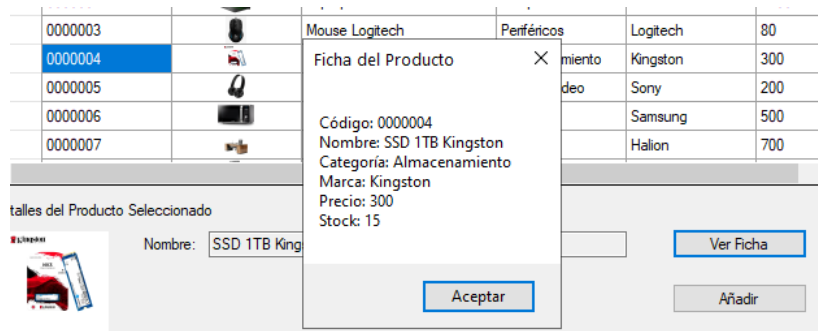


Funcionalidad 7: Mostrar ficha del producto

Se presenta información detallada en formato textual.

Muestra todos los datos del producto.

```
private void btnFicha_Click(object sender, EventArgs e)
{
    if (dataGridView1.CurrentRow == null) return;
    var p = (Producto)dataGridView1.CurrentRow.DataBoundItem;
    MessageBox.Show(
        $"Código: {p.Codigo}\nNombre: {p.Nombre}\nCategoría: {p.Categoría}\nMarca: {p.Marca}\nPrecio: {p.Precio}\nStock: {p.Stock}",
        "Ficha del Producto"
    );
}
```

Funcionalidad 8: Añadir producto

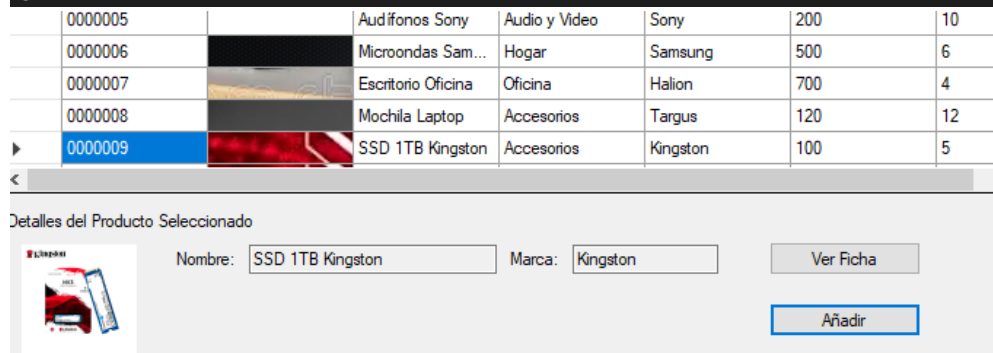
Permite ampliar el catálogo dinámicamente.

Genera un código automático y agrega el producto a la lista. También asigna una imagen si existe.

```
private void btnAñadir_Click(object sender, EventArgs e)
{
    Producto nuevo = new Producto()
    {
        Codigo = (listaProductos.Count + 1).ToString("0000000"),
        Nombre = txtNombre.Text,
        Marca = txtMarca.Text,
        Categoría = "Accesorios",
        Precio = 100,
        Stock = 5,
        Imagen = pictureBox1.Image != null ? (Image)pictureBox1.Image.Clone() : SystemIcons.Question.ToBitmap()
    };

    listaProductos.Add(nuevo);

    dataGridView1.DataSource = null;
    dataGridView1.DataSource = listaProductos;
}
```



IV. ANALISIS E INTERPRETACION DE RESULTADOS

¿Qué indican los resultados?

- El sistema gestiona productos de manera eficiente.
- La búsqueda funciona adecuadamente combinando texto y categoría.
- La visualización de imágenes mejora la usabilidad del sistema.

¿Que se ha encontrado?

- LINQ permite el filtrado de datos complejos.



- El uso de imágenes en DataGridView mejora la interfaz.
- La interacción mediante selección mejora la usabilidad.
- La estructura del sistema permite añadir nuevas funcionalidades fácilmente.

CONCLUSIONES

- Se ha conseguido crear un sistema completo de catálogo de productos con varias funciones.
- El uso de listas, LINQ y DataGridView supuso una forma adecuada de gestionar datos.
- El tratamiento de imágenes dio un valor visual a la aplicación.
- La búsqueda combinación confianza demostró ser flexible y útil para el usuario.
- Se ha conseguido la programación orientada a objetos en las partes de los datos.
- El sistema es escalable, y a partir de este se puede evolucionar aplicaciones complejas como tiendas virtuales.

RECOMENDACIONES

- Implementar almacenamiento en base de datos.
- Permitir la edición y eliminación de productos.
- Añadir carga de imágenes por parte del usuario.

WEBGRAFIA

BillWagner. (s. f.). Operadores y expresiones: enumerar todos los operadores y expresiones - C# reference. Microsoft Learn. Recuperado de <https://learn.microsoft.com/es-es/dotnet/csharp/language-reference/operators/>

Arreglos en C#. (2014). C# Paso a Paso. Recuperado de <https://csharp-facilito.blogspot.com/2013/07/arreglos-en-c-sharp.html>

Turrado, J. (2022). Introducción rápida a LINQ con C#: manejar información en memoria nunca fue tan sencillo - campusMVP.es. campusMVP.es. Recuperado de https://www.campusmvp.es/recursos/post/introduccion-rapida-a-linq-con-c-sharp.aspx?srsId=AfmBOoqx5_WZq2mtHdV0_YFJiBZsZ5dj_t3Us87MablaeW1k-0upk2xK